Expensify Lite

Project Report — Flask and MySQL Expense Tracker

1. Problem statement

Recording daily expenses manually can make it difficult to find transactions, correct mistakes, compare categories, or understand spending patterns. Expensify Lite is a local web application that stores expense records in MySQL and presents filtered records and monthly summaries through a Flask interface.

2. Objectives

• Record expenses with amount, description, category, date, and payment method.

• Support expense creation, editing, deletion, searching, filtering, and sorting.

• Calculate monthly totals, counts, average expense amounts, category breakdowns, and a six-month trend.

• Provide CSV export and JSON endpoints for analytics.

3. Technology stack

Technology	Use in the project
Python	Application logic, validation, date/currency formatting, CSV generation
Flask	Routes, request handling, templates, redirects, flash messages, JSON responses
MySQL Connector/Python	Connection to MySQL and execution of parameterized SQL
MySQL	Persistent storage, indexes, CRUD operations, aggregate queries
HTML/CSS/JavaScript/Chart.js	Browser interface and charting layer, as indicated by the project stack

4. Architecture and methodology

The application is separated into two main Python modules. **app.py** defines the Flask app and routes. It reads form values and query parameters, validates monetary input, calls the database module, and returns templates, redirects, CSV, or JSON. **database.py** holds database configuration, schema initialization, validation, CRUD operations, filters, and analytics queries.

Database connections are created through get_connection(). SQL values are passed as parameters. Insert, update, and delete operations commit when successful and roll back if an exception occurs. Connections and cursors are closed in finally blocks.

5. Database design

The init_db() function creates an expenses table and an app_meta table if they do not already exist.

Column	Type	Purpose
id	INT AUTO_INCREMENT PRIMARY KEY	Unique record identifier
amount_paise	BIGINT NOT NULL	Money stored as integer paise
description	VARCHAR(255) NOT NULL	Expense description
category	VARCHAR(50) NOT NULL	Expense category
expense_date	VARCHAR(10) NOT NULL	Date in YYYY-MM-DD format
payment_method	VARCHAR(50) NOT NULL	Payment method
created_at	VARCHAR(30) NOT NULL	Creation timestamp

The table includes indexes on expense_date and category. Database settings are read from MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, and MYSQL_DATABASE environment variables.

6. Implementation details

Money handling: parse_amount_to_paise() strips the rupee symbol and commas, validates the input, and converts rupees to integer paise. This keeps stored monetary values in integer form. format_inr() formats values using Indian digit grouping.

Validation: the database layer checks that the amount is a positive integer, the description is not blank, the category and payment method are in their configured lists, and the date uses YYYY-MM-DD format.

Expense operations: add_expense(), update_expense(), delete_expense(), get_expense(), and get_expenses() implement create, update, delete, single-record retrieval, and list retrieval. Listing supports description search, category, date range, sort order, and an optional limit.

Analytics: get_monthly_summary() calculates monthly count, sum, and average. get_category_breakdown() groups records by category and sorts by total spending. get_six_months_trend() calculates totals and counts for six months.

Export and APIs: /expenses/export produces CSV using the current filters. /api/analytics/category and /api/analytics/trend return analytics as JSON.

Request protection: app.py creates a session CSRF token and verifies it on the add, edit, and delete POST routes.

7. Main routes

Route	Purpose
/	Redirects to dashboard
/dashboard	Current-month summary, recent expenses, category breakdown, total count
/expenses	Search, filters, sorting, and filtered total
/expenses/add	Add a validated expense
/expenses/edit/<id>	Update an expense
/expenses/delete/<id>	Delete an expense
/expenses/export	Export filtered records to CSV
/analytics	Monthly analytics and six-month trend
/api/analytics/category	Category breakdown as JSON
/api/analytics/trend	Six-month trend as JSON

8. Limitations

• Dates are stored as VARCHAR values in YYYY-MM-DD format rather than a native MySQL DATE column.

• The six-month trend runs one aggregate query per month.

• Authentication and per-user data separation are not shown in the supplied app.py and database.py.

• The interface details and browser-side chart implementation depend on templates and static files not included in the two source files documented here.

9. Conclusion

Expensify Lite combines Flask request handling with MySQL persistence to provide expense management and spending summaries. Its implementation demonstrates Python functions and exception handling, input validation, web routing, parameterized SQL, transaction handling, filtering, aggregation, JSON responses, and CSV generation. The companion notebook offers a repeatable walkthrough using the project's configured database.